



Errata

CAPÍTULO 2

Página 59

Tabela 2.7 - Operadores aritméticos

Operador	Operação	Tipo	Resultado
+	Manutenção de sinal	Unário	-
-	Inversão de sinal	Unário	-
%	Resto de divisão (módulo)	Binário	Inteiro
/	Divisão	Binário	Inteiro ou Real
*	Multiplificação	Binário	Inteiro ou Real
+	Adição	Binário	Inteiro ou Real
-	Subtração	Binário	Inteiro ou Real
++	Incremento	Unário	Inteiro
--	Decremento	Unário	Inteiro
+=	Adição cumulativa	Binário	Inteiro ou Real
-=	Subtração cumulativa	Binário	Inteiro ou Real
*=	Multiplificação cumulativa	Binário	Inteiro ou Real
/=	Divisão cumulativa	Binário	Inteiro ou Real
%=	Módulo cumulativo	Binário	Inteiro ou Real
pow(b, e)	Exponenciação	Binário	Real

Operador	Operação	Tipo	Resultado
cbrt(n)	Raiz Cúbica	Unário	Real
sqrt(n)	Raiz quadrada	Unário	Real

Página 73

```

1 // C02EX05.CPP
2
3 #include <iostream>
4 #include <locale> // para setlocale()
5 #include <windows.h> // para SetConsoleCP()
6 #include <string> // para usar std::wstring
7
8 using namespace std;
9
10 int main(void)
11 {
12     wstring nome;
13
14     setlocale(LC_ALL, "Portuguese_Brazil.1252");
15     SetConsoleCP(1252);
16
17     wcout << L"Olá.\nQual sua graça? ";
18     getline(wcin, nome);
19
20     cout << endl;
21     wcout << L"Oi " << nome << L", vamos estudar." << endl;
22     cout << endl;
23
24     cout << "Tecle <Enter> para encerrar... ";
25     cin.get();
26     return 0;
27 }

```

No programa **C02EX05**, o código é similar ao do programa anterior, mas com a adição de configurações específicas para garantir a correta manipulação de caracteres acentuados. A configuração da localidade e da página de código do console é essencial para que o programa funcione corretamente em sistemas Windows.

As linhas 3 a 6 do programa incluem as bibliotecas necessárias para realizar operações de entrada e saída, configurar a localidade do programa, manipular configurações específicas do sistema operacional Windows e trabalhar com cadeias de

caracteres largos. Essas inclusões são essenciais para garantir que o programa funcione corretamente e manipule caracteres acentuados de forma adequada.

Na linha 9, a variável **nome** é declarada como **wstring**. Diferentemente do programa anterior que usou **string**, neste caso, **wstring** é usado para armazenar cadeias de caracteres largos (*wide characters*), que são necessários para manipular caracteres acentuados e outros caracteres especiais. A classe **wstring** é uma versão da classe **string** que suporta caracteres largos, proporcionando uma maneira mais segura e conveniente de manipular cadeias de caracteres com caracteres acentuados.

Na linha 10, a função **setlocale()** é usada para configurar a localidade do programa. Esta função ajusta a localidade para corresponder à configuração regional do Brasil, especificando que o idioma é o português e a codificação de caracteres é a **Windows-1252**. O primeiro argumento, **LC_ALL**, indica que todas as categorias de localidade devem ser configuradas, enquanto o segundo argumento, **Portuguese_Brazil.1252**, especifica a localidade desejada e a codificação de caracteres. A codificação **Windows-1252** é uma extensão do *Latin-1* e é frequentemente utilizada em sistemas Windows para suportar caracteres especiais do português.

A linha 11, por meio de **SetConsoleCP(1252);**, configura a página de código para a entrada do console para **1252**, garantindo que os caracteres acentuados sejam lidos corretamente do teclado.

Na linha 14, a função **getline(wcin, nome);** é utilizada para ler uma linha completa de texto da entrada padrão e armazená-la na variável **nome**. Esta função é especialmente útil quando se deseja capturar uma entrada do usuário que pode conter espaços, ao contrário do operador **>>** que lê apenas até o primeiro espaço. A função **getline** lê caracteres até encontrar um caractere de nova linha **"\n"**, garantindo que toda a linha seja capturada corretamente.

Na linha 16, a função **wcout** é utilizada para exibir a saída. A saída inclui a cadeia **"Oi "** seguida do conteúdo da variável **nome** e a cadeia **", vamos estudar."**. A função **wcout** é a versão de **cout** que suporta caracteres largos, garantindo que os caracteres acentuados sejam exibidos corretamente no console.

É importante notar que, apesar da configuração de localidade e página de código estarem corretas, a exibição de caracteres acentuados pode não funcionar como esperado no Windows devido a diversas configurações e particularidades do sistema operacional. Isso pode ocorrer se o sistema não estiver configurado adequadamente para suportar a codificação especificada ou se houver outros fatores relacionados às configurações regionais. Caso surjam problemas com a exibição

de caracteres acentuados, pode ser necessário ajustar as configurações do sistema ou buscar soluções adicionais em outras fontes.

CAPÍTULO 3

Página 109/110

```
1 // C03EX03.CPP
2
3 #include <iostream>
4 using namespace std;
5
6 int main(void)
7 {
8
9     int32_t a, b, x;
10
11     cout << "Entre o primeiro valor ..: "; cin >> a;
12     cin.ignore(80, '\n');
13
14     cout << "Entre o segundo valor ...: "; cin >> b;
15     cin.ignore(80, '\n');
16
17     cout << "\n";
18     if (a > b)
19     {
20         x = a;
21         a = b;
22         b = x;
23     }
24
25     cout << "Os valores ordenados sao: " << a;
26     cout << " e " << b << "." << endl << endl;
27
28     cout << "Tecle <Enter> para encerrar... ";
29     cin.get();
30     return 0;
31 }
32 }
```

```
1 // C03EX05.CPP
2
3 #include <iostream>
4 #include <ciso646> // opcional no GCC
5
6 using namespace std;
7
8 int main(void)
9 {
10     int32_t numero;
11
12     cout << "Entre um valor: "; cin >> numero;
13     cin.ignore(80, '\n');
14     cout << endl;
15
16     if (numero >= 20 and numero <= 90)
17         cout << "O valor esta entre 20 e 90.";
18     else
19         cout << "O valor nao esta entre 20 e 90.";
20
21     cout << "\n" << endl;
22
23     cout << "Tecle <Enter> para encerrar... ";
24     cin.get();
25     return 0;
26 }
```

